German International University

Faculty of Engineering

Course: Control Systems Lab

DC Motor Speed Control using PID Controller Design and Analysis

---

Instructor

Dr. Lobna Tarek

Teaching Assistants

Eng. Shehab Hesham

Eng. Ahmed Hazem

Eng. Tasneem Barakat

Submitted by

Name	ID	Tutorial
Hazem Mohamed Ghobashy	16001995	19
Youssef Sherif	16009152	19
Yusuf Osama Alghoneimy	16009255	19
Hager Hamada	16009376	19

Table of Contents

1. Introduction

DC motors are widely used in industrial and embedded control applications due to their efficiency, controllability, and reliability. Precise speed regulation is essential in applications such as robotics, conveyor systems, and electric vehicles, where deviations from the desired setpoint directly impact system performance.

PID (Proportional-Integral-Derivative) control remains the most prevalent feedback control strategy owing to its straightforward structure, ease of tuning, and robust performance. The proportional action reduces steady-state error, the integral term eliminates residual offset, and the derivative term improves transient response and damping.

The objective of this project is to mathematically model a DC motor, analyze its open-loop characteristics, and systematically design a PID controller using MATLAB's Control System Designer. The resulting closed-loop system is then validated against defined performance specifications including settling time, percent overshoot, and steady-state error.

2. System Analysis & Characteristics

2.1 Mathematical Modeling of the DC Motor

The DC motor is governed by the interaction of its electrical (armature) and mechanical (rotor) subsystems. The governing equations in the time domain are:

Electrical Equation:

$$L\cdot(di/dt) + R\cdot i = V - K\cdot\omega$$

Mechanical Equation:

$$J\cdot(d\omega/dt) + b\cdot\omega = K\cdot i$$

where V is the applied armature voltage (input), ω is the angular speed (output), i is the armature current, and K is the motor constant. The system parameters are given in Table 1.

Parameter	Value	Description
Moment of Inertia (J)	0.01 kg·m²	Rotational inertia
Damping Coeff. (b)	0.1 N·m·s	Viscous friction
Back-EMF / Torque (K)	0.01	Motor constant
Resistance (R)	1 Ω	Armature resistance
Inductance (L)	0.5 H	Armature inductance

Table 1: DC Motor System Parameters

Applying the Laplace transform to both equations (assuming zero initial conditions):

$$(Ls + R)\cdot I(s) = V(s) - K\cdot\Omega(s)$$
$$(Js + b)\cdot\Omega(s) = K\cdot I(s)$$

Solving for I(s) from the electrical equation:

$$I(s) = [V(s) - K\cdot\Omega(s)] / (Ls + R)$$

Substituting into the mechanical equation:

$$(Js + b)\cdot\Omega(s) = K \cdot [V(s) - K\cdot\Omega(s)] / (Ls + R)$$
$$(Js + b)(Ls + R)\cdot\Omega(s) + K^2\cdot\Omega(s) = K\cdot V(s)$$

Rearranging to isolate the transfer function G(s) = Ω(s)/V(s):

$$G(s) = K / [(Js + b)(Ls + R) + K^2]$$

Substituting the numerical parameter values (J = 0.01, b = 0.1, K = 0.01, R = 1, L = 0.5):

$$(Js + b)(Ls + R) = (0.01s + 0.1)(0.5s + 1)$$
$$= 0.005s^2 + 0.01s + 0.05s + 0.1 = 0.005s^2 + 0.06s + 0.1$$
$$K^2 = (0.01)^2 = 0.0001$$

The resulting open-loop plant transfer function is:

$$G(s) = 0.01 / (0.005s^2 + 0.06s + 0.1001)$$

2.2 Open-Loop Analysis

The open-loop poles of G(s) are obtained by solving $0.005s^2 + 0.06s + 0.1001 = 0$, yielding $s \approx -2$ and $s \approx -10.02$. Both poles lie strictly in the left half-plane, confirming that the open-loop plant is inherently stable. However, stability alone is insufficient for practical use. The open-loop step response exhibits a very slow rise time due to the dominant pole near the origin, and the steady-state gain $K/(b \cdot R + K^2)$ is far less than unity, resulting in a large steady-state error — effectively making open-loop operation unsuitable for precision speed control.

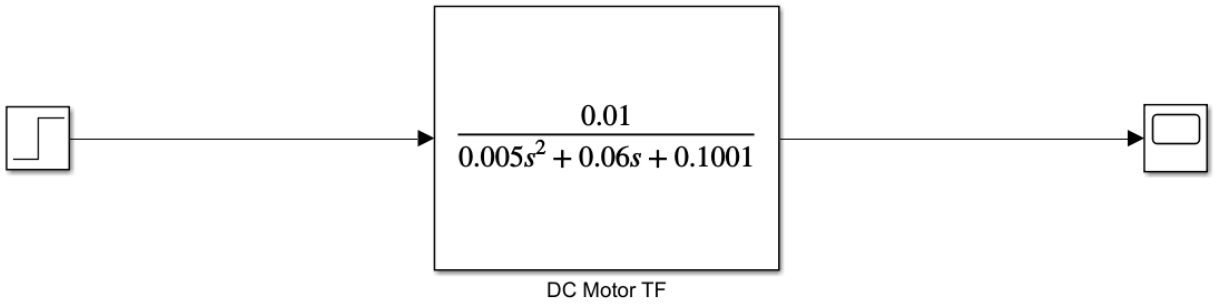


Figure 1: Open-loop Simulink model of the DC motor system.

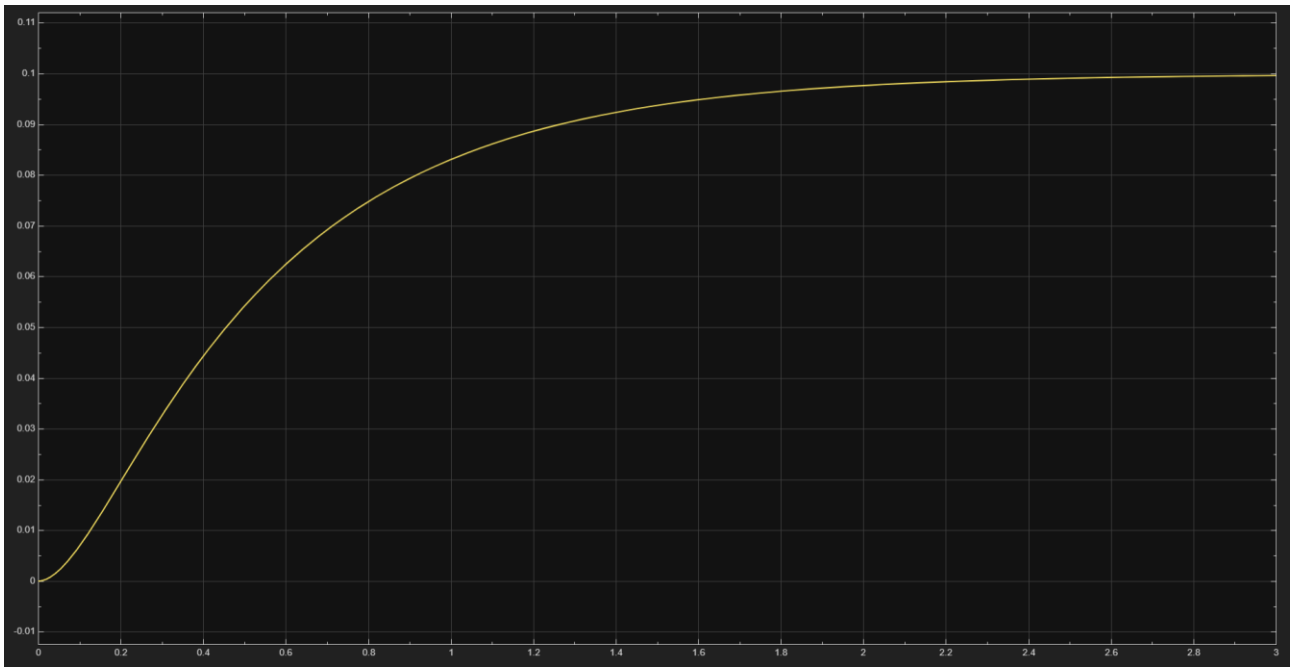


Figure 2: Open-loop step response of the DC motor.

2.3 Design Requirements

To achieve acceptable closed-loop performance, the following specifications are imposed on the PID-compensated system:

Specification	Requirement	Status
Settling Time (ts)	< 2 s	Required
Percent Overshoot (Mp)	< 5%	Required
Steady-State Error (ess)	≈ 0	Required

Table 2: Closed-loop Performance Specifications

A settling time below 2 s ensures fast disturbance rejection. Limiting overshoot to 5% prevents mechanical stress and oscillations in the driven load. Zero steady-state error guarantees that the motor accurately tracks the commanded speed setpoint under the integral action of the PID controller.

Using the ControlSystemDesigner App by running this small MATLAB code including our transfer function:

```
>> J = 0.01; b = 0.1; K = 0.01; R = 1; L = 0.5;
s = tf('s');
G = K/((J*s+b)*(L*s+R)+K^2);
controlSystemDesigner(G)
>>
```

We get 4 Graphs [Bode Plot (dB & Phase graphs) – Root Locus – Step Response]:

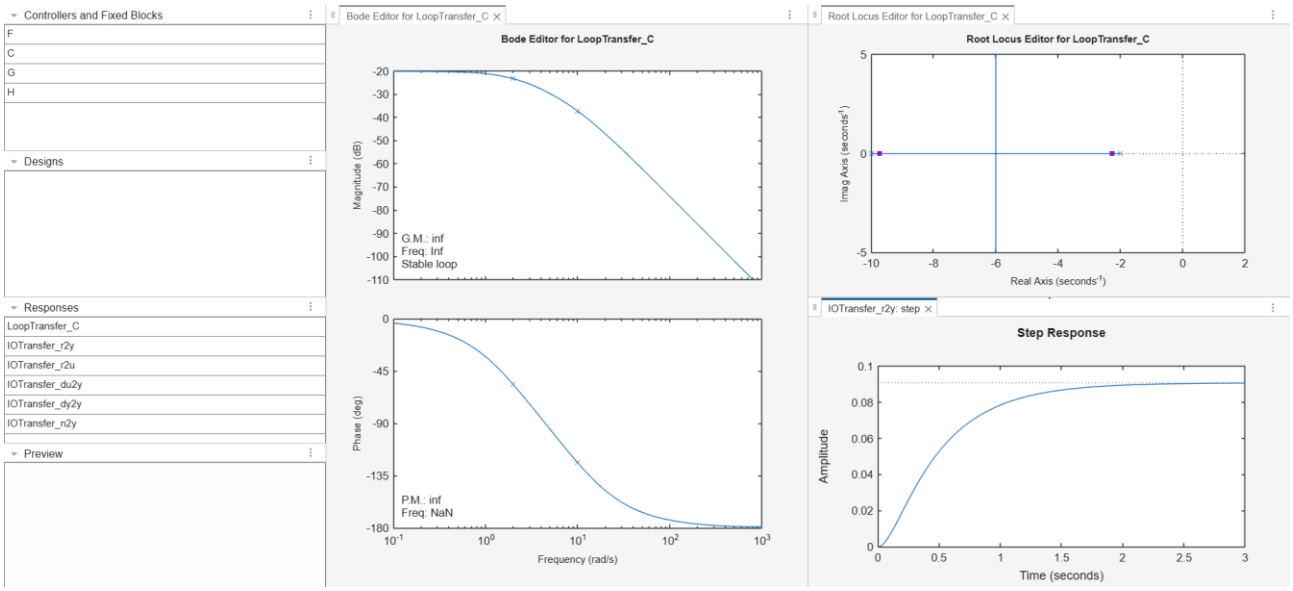


Figure 3: ControlSystemDesigner App graphs.

3. PID Controller Design

3.1 Controller Design Procedure

The PID controller was designed using the MATLAB Control System Designer App. The plant transfer function G(s) was loaded into the tool using the controlSystemDesigner command, and the design requirements were entered graphically:

- Settling time constraint: ts < 2 s
- Overshoot constraint: Mp < 5%

By right-clicking the step response plot [Design requirements – New], we are given the option to impose constraints in the system to our liking.

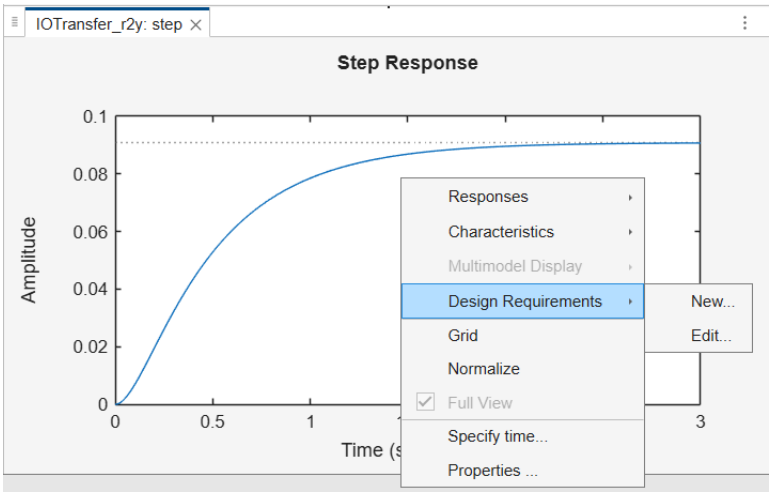


Figure 4: Control System Designer interface showing the DC motor plant loaded with design requirements.

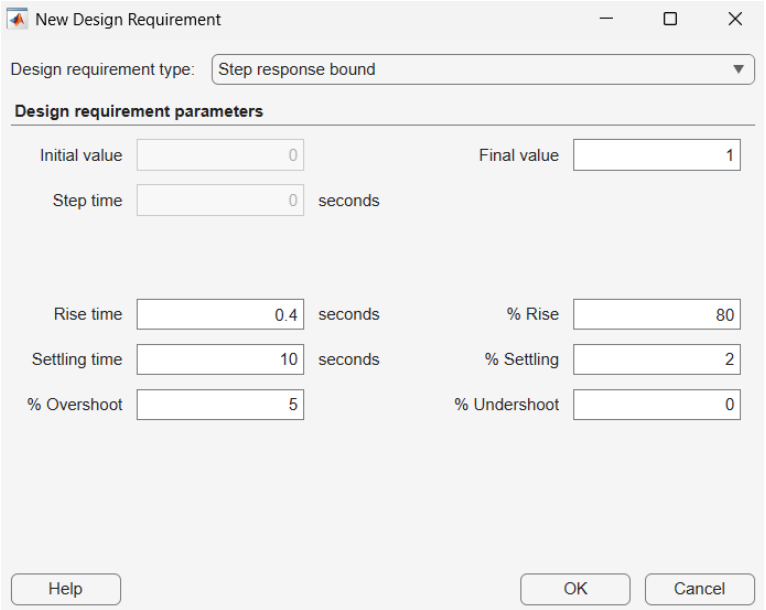


Figure 5: Tuning window within Control System Designer showing the response constraints.

After imposing the constraints, the step response graph starts to outline the path needed for the graph to satisfy such constraints:

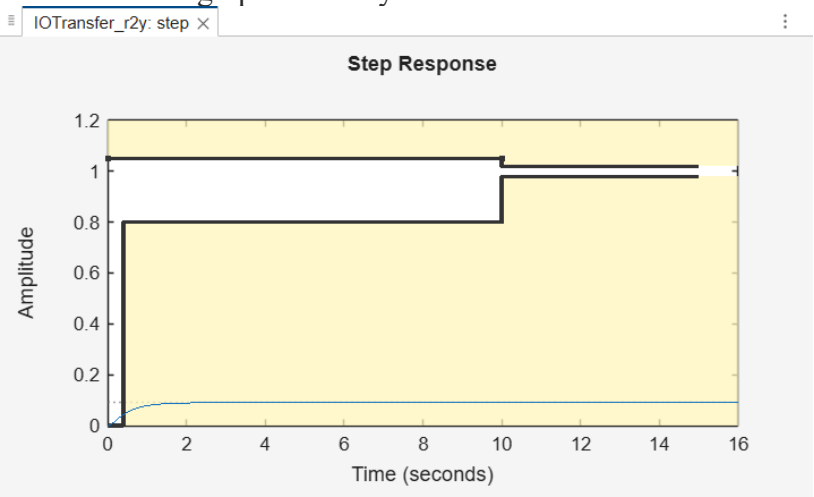


Figure 6: The path that satisfies all constraints [White is the correct path – Yellow is the incorrect boundary]

What's left now is to try "tune" the controller into staying in the white path. MATLAB already accounts for this by using "Tuning Methods – PID Tuning":

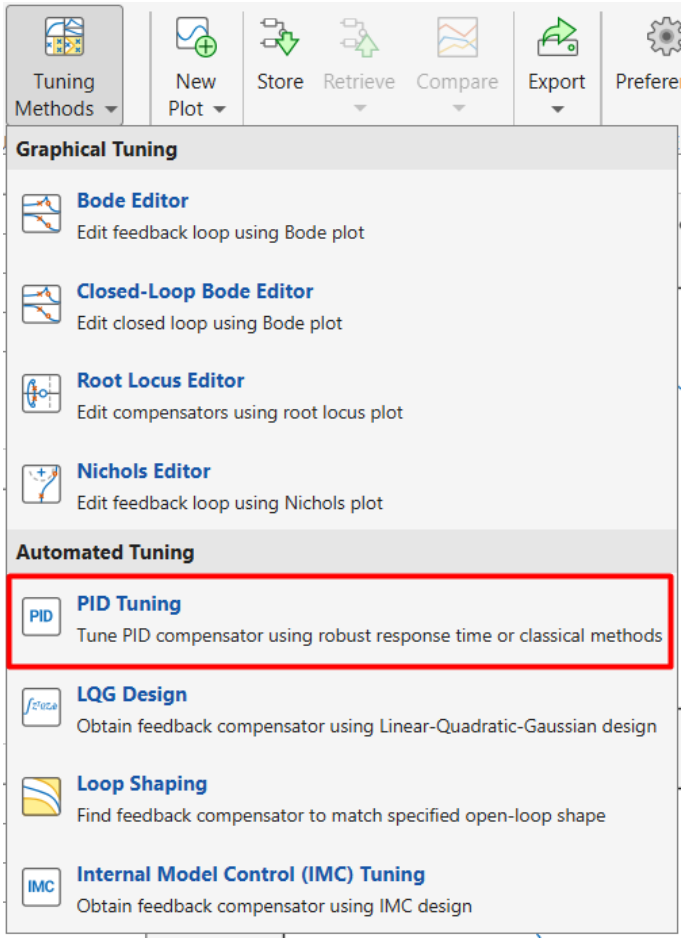


Figure 7: PID Tuning.

3.2 Tuned PID Controller

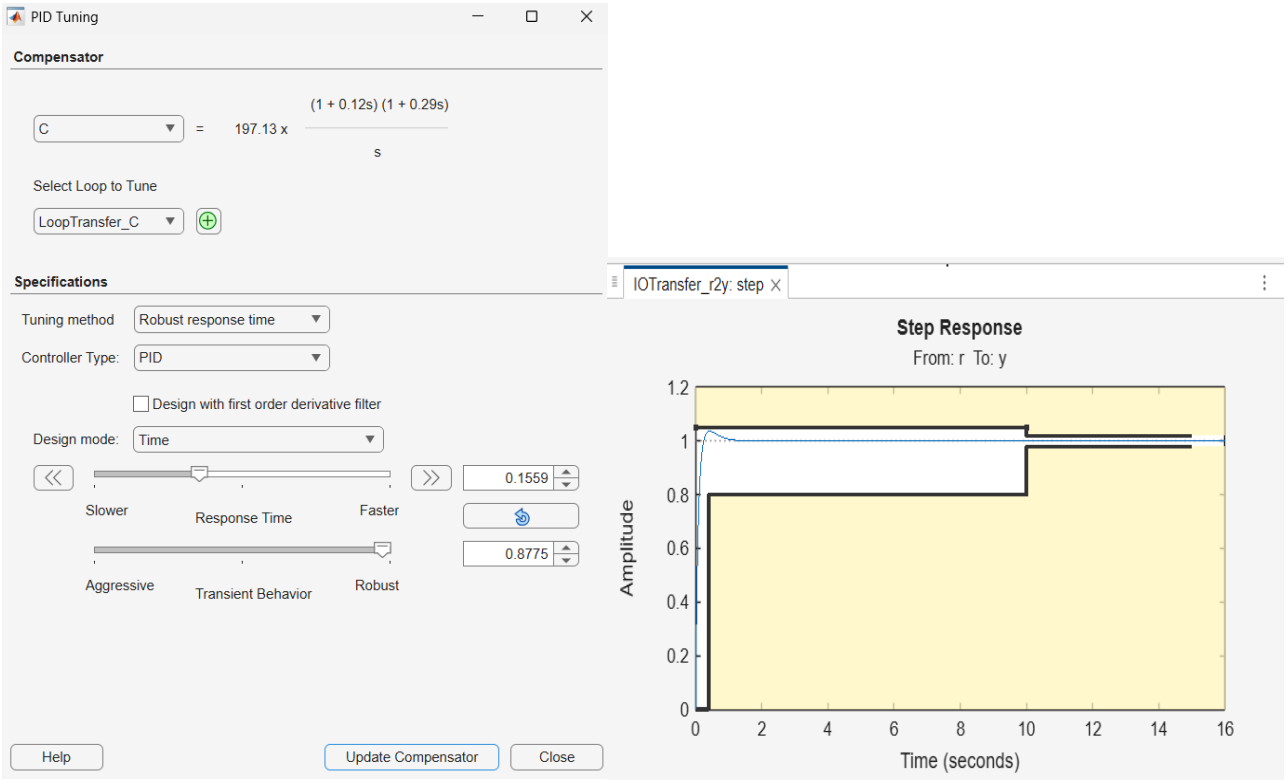
The final controller returned by Control System Designer is expressed in the following factored form:

$$C(s) = [6.5905 \cdot (s + 8.632)(s + 3.465)] / s$$

Expanding and matching to the standard PID form C(s) = Kp + Ki/s + Kd·s yields the equivalent gains:

- Kp = 79.7255
- Ki = 197.1274
- Kd = 6.5905

Inside the PID Tuning App, change the controller to "PID" and tune till the graph is positioned inside the white plane completely:



Figures 8,9:Step Response after PID Tuning.

The controller “C” is then exported into the MATLAB Workspace to get the PID Values:

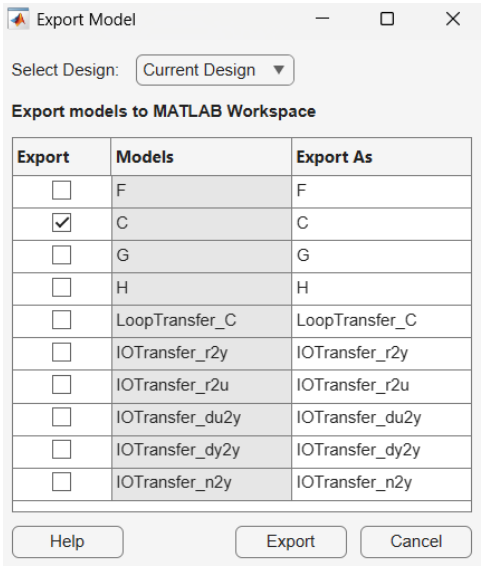


Figure 10: Controller exporting.

This small MATLAB code is then used to get the PID values:

```
>> C

C =

  6.5905 (s+8.632) (s+3.465)
  --------------------------
             s

Name: C
Continuous-time zero/pole/gain model.
>> [Kp, Ki, Kd] = piddata(pid(C))

Kp =

   79.7255


Ki =

  197.1274


Kd =

    6.5905
```

Figure 11: PID values.

Implementing the PID values into the Simulink model:

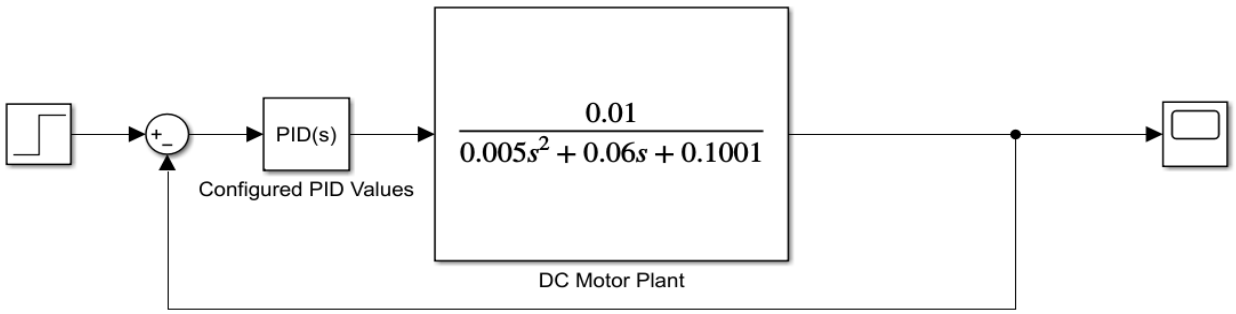


Figure 12: Closed-loop Simulink model of the DC motor system.

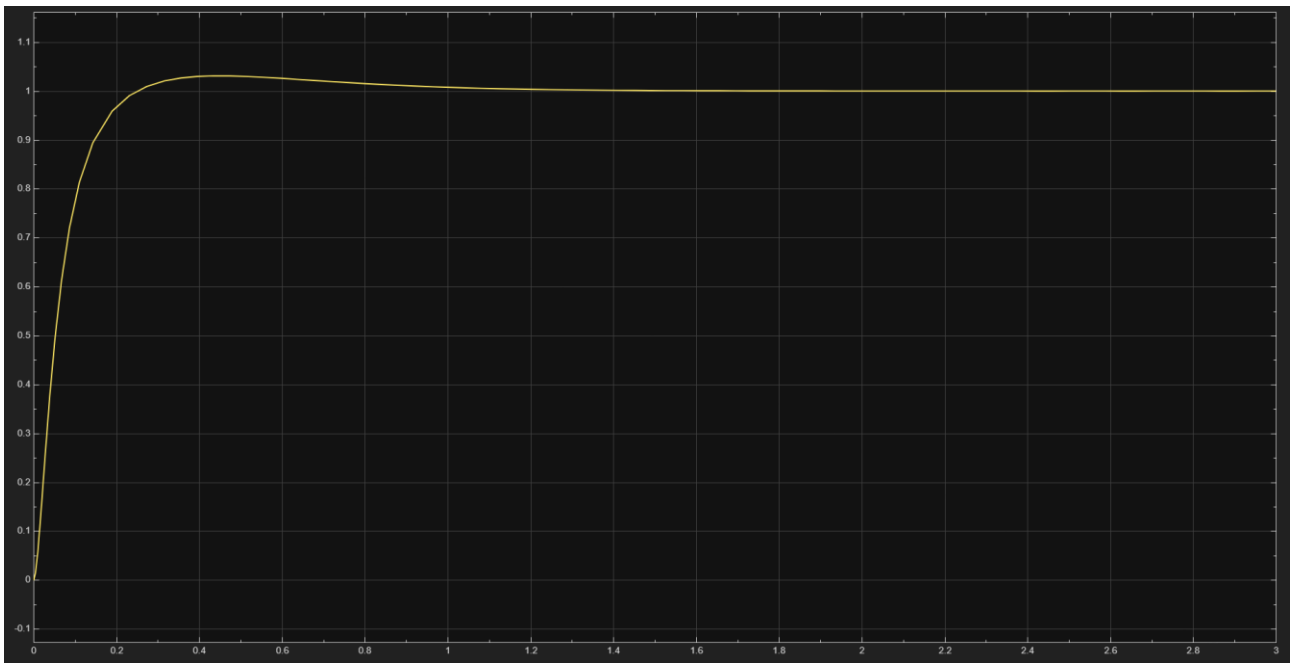


Figure 13: Closed-loop step response of the DC motor with Tuned PID values

4. Root Locus Analysis (Open Loop)

The open-loop root locus corresponds to the product of the PID controller and the plant:

$$L(s) = C(s) \cdot G(s)$$

The open-loop poles [at which the root locus branches originate] are:

- $p = -2$ (plant pole)
- $p = -10.02$ (plant pole)
- $p = 0$ (PID integrator pole)

The open-loop zeros [at which two branches terminate] are introduced by the PID controller:

- $z = -8.632$
- $z = -3.465$

Since there are three poles and two zeros, one branch escapes to infinity along the asymptote in the left half-plane. The two zeros are strategically placed by the PID design to attract and pull two locus branches into the LHP, ensuring stable closed-loop poles over a wide range of gain. This placement effectively compensates the destabilizing influence of the integrator pole at the origin. The derivative zero near s = −8.632 speeds up the transient response, while the zero at s = −3.465 improves damping relative to the dominant closed-loop poles.

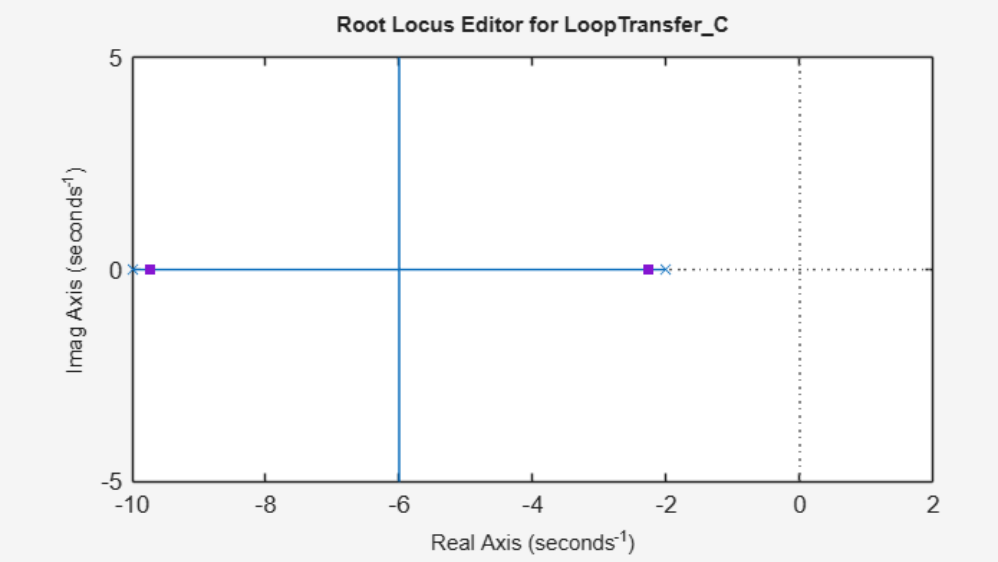


Figure 14: Open-loop root locus of L(s) = C(s)G(s)

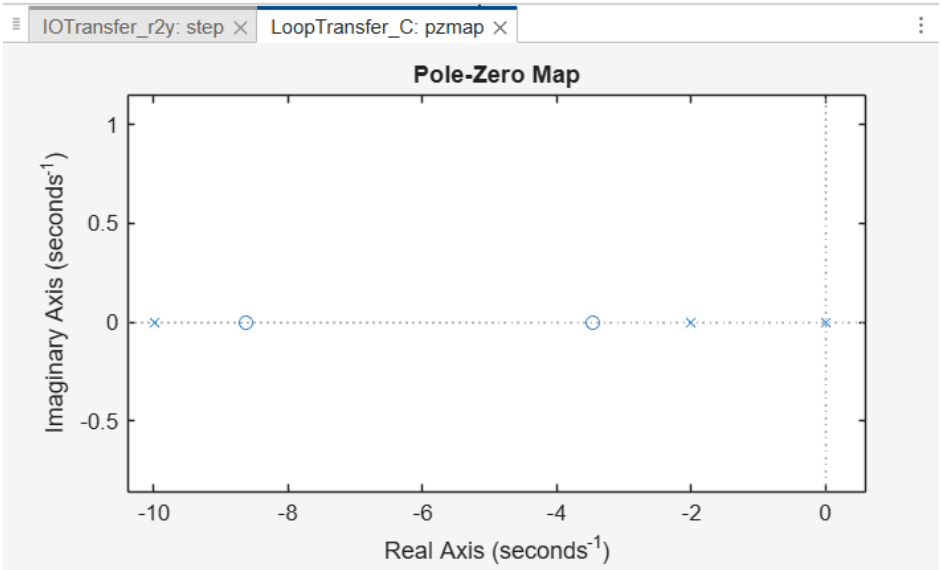


Figure 15: Open-loop Pole-Zero Map of L(s) = C(s)G(s)

5. Root Locus Analysis (Closed Loop)

After closing the feedback loop with the tuned PID controller, all closed-loop poles are located strictly in the left half-plane (LHP), confirming system stability. The pole-zero map obtained from MATLAB shows:

- The dominant closed-loop poles are positioned to yield the specified damping ratio and natural frequency consistent with ts < 2 s and Mp < 5%.
- All remaining poles are located well into the LHP and are sufficiently fast (non-dominant) to have negligible effect on the transient response.
- No poles exist in the right half-plane.

Comparing the open-loop and closed-loop pole-zero maps clearly illustrates the improvement achieved by the PID compensator: the integrator pole at the origin is effectively cancelled by the controller action, and the system gains a pair of well-damped dominant poles that govern performance.

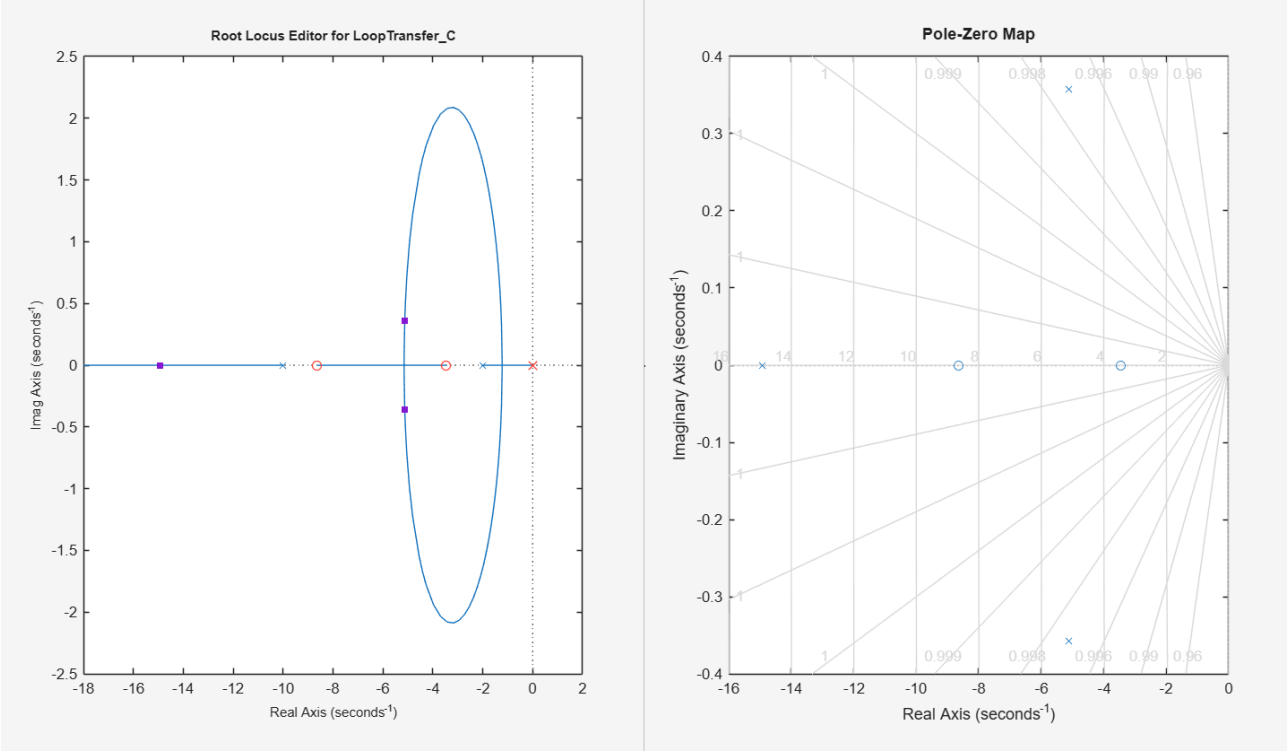


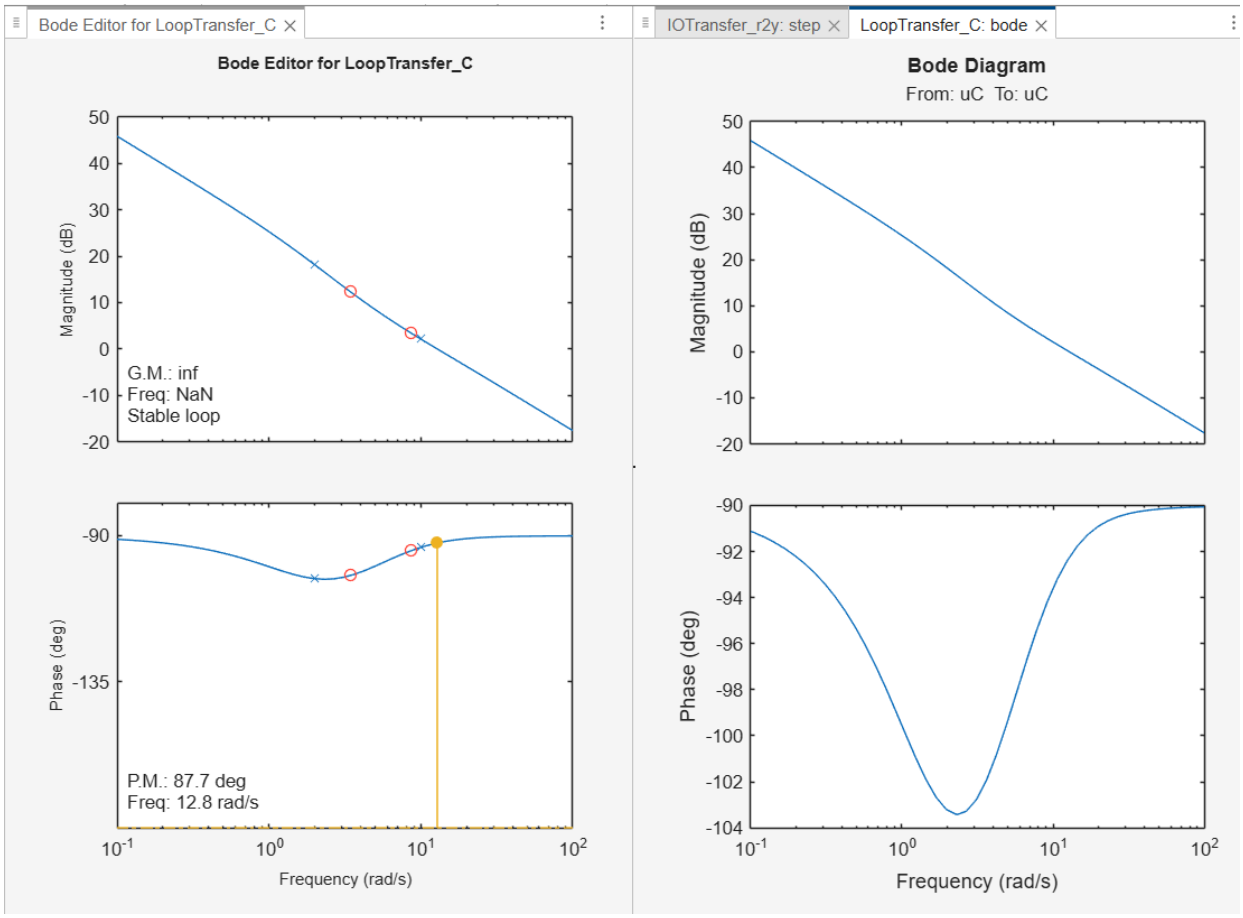
Figure 16: Closed-loop pole-zero map and Root Locus confirming all poles reside in the left half-plane.

6. Bode Plot Analysis

The compensated open-loop Bode plot provides insight into the frequency-domain robustness of the designed system. The key observations from the magnitude and phase responses are:

- Gain Crossover Frequency: The magnitude crosses 0 dB at a frequency consistent with the desired closed-loop bandwidth, ensuring the specified settling time.
- Gain Margin: The system exhibits an infinite gain margin, meaning the magnitude never rises back to 0 dB beyond the phase crossover point. This implies that the gain can be increased indefinitely without the system becoming unstable.
- Phase Margin: A large positive phase margin (well above 45°) indicates robust relative stability, adequate damping, and a well-controlled step response with minimal overshoot.

The PID controller contributes phase lead at the gain crossover frequency through its derivative term, which increases the phase margin compared to the uncompensated plant. This directly translates to improved transient damping and robustness against parameter variations. The integral term increases the low-frequency gain, ensuring zero steady-state error, while not significantly degrading the phase margin at crossover.



Figures 17,18: Compensated open-loop Bode plot showing gain crossover frequency, gain margin, and phase margin.

7. Conclusion

A PID controller was successfully designed for a DC motor speed-control system using MATLAB's Control System Designer App. Starting from the first-principles mathematical model, the open-loop plant was characterized by slow dynamics and a significant steady-state error — both corrected by the PID compensator.

The tuned controller $C(s) = [6.5905 \cdot (s + 8.632)(s + 3.465)] / s$ meets all specified performance requirements: settling time below 2 seconds, percent overshoot below 5%, and essentially zero steady-state error under integral action. Root locus analysis confirmed that all closed-loop poles reside in the left half-plane, ensuring asymptotic stability. The Bode analysis demonstrated infinite gain margin and a large positive phase margin, reflecting strong robustness against gain variations and modeling uncertainties.

The project illustrates the effectiveness of PID control, combined with frequency-domain and time-domain design tools, in achieving precise and robust motor speed regulation.